



دانشکده علوم پایه
گروه علوم کامپیوتر
پروژه کارشناسی

عنوان

نبرد (نرم افزار برنامه ریزی راهبردی دانشگاه)

استاد راهنما
دکتر کشاورز

دانشجو
امیرحسین رجبی

بهار 1404

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

3.....	چکیده:
4.....	فصل اول-مقدمه
5.....	1-1. مقدمه
5.....	1-2. بیان مسئله
6.....	1-3. هدف پروژه نبرد
6.....	1-4. ابزارها و فناوری‌های مورد استفاده
7.....	1-5. دستاوردها و مزایا
8.....	فصل دوم-مفاهیم پایه و کارهای مرتبط
9.....	2-1. مقدمه
9.....	2-2. مفاهیم پایه
9.....	2-2-1. برنامه‌ریزی و زمان‌بندی آموزشی در دانشگاه‌ها
9.....	2-2-2. معماری کلاینت-سرور (Client-Server Architecture)
10.....	2-2-3. React.js و رابط کاربری
10.....	2-2-4. Ant Design
10.....	2-2-5. Express.js و Node.js
10.....	2-2-6. Prisma ORM
11.....	2-2-7. MySQL
11.....	2-2-8. Docker و Docker Compose
11.....	2-2-9. مفاهیم برنامه‌ریزی هوشمند و بهینه‌سازی زمان‌بندی
12.....	2-3. کارهای مرتبط
12.....	2-3-1. روش‌های سنتی (Excel و فرم‌های دستی)
12.....	2-3-2. نرم‌افزارهای مدیریت آموزشی موجود
12.....	2-3-3. مقایسه نبرد با راهکارهای موجود
13.....	2-3-4. مزیت رقابتی نبرد
13.....	2-4. جمع‌بندی فصل
14.....	فصل سوم-طراحی و پیاده‌سازی سیستم
15.....	3-1. مقدمه
15.....	3-2. طراحی پایگاه داده
17.....	توضیح جداول و روابط
18.....	3-3. طراحی و پیاده‌سازی بک‌اند
18.....	3-3-1. مسیرها (Routes)

20.....	3-3-2. کنترلرها ((Controllers
25.....	3-3-3. سرویس‌ها ((Services
26.....	3-3-4. اعتبارسنجی داده‌ها (Validation) با Zod
26.....	۳-۴. طراحی و پیاده‌سازی فرانت‌اند
27.....	۳-۴-۱. ساختار پوشه‌ها
27.....	۳-۴-۲. صفحه پنل اساتید ((ProfessorPanel.tsx
31.....	۳-۴-۳. فرم افزودن/ویرایش استاد ((ProfessorForm.tsx
32.....	۳-۴-۵. جمع‌بندی
33.....	3-5. زیرساخت و دیپلوی
33.....	3-6. جمع‌بندی فصل
34.....	فصل چهارم- حل مسئله و الگوریتم زمان‌بندی
35.....	۴-۱. مقدمه
35.....	۴-۲. تعریف دقیق مسئله
35.....	۴-۳. بررسی الگوریتم‌های موجود
36.....	۴-۴. انتخاب راه حل ما: Backtracking + MRV
36.....	۴-۵. تحلیل خط‌به‌خط تابع generatePlan
38.....	۴-۶. خروجی الگوریتم
38.....	۴-۷. مزایا و قابلیت توسعه الگوریتم
38.....	۴-۸. جمع‌بندی
39.....	فصل پنجم- نتیجه‌گیری و پیشنهادات
40.....	۵-۱. مقدمه
40.....	۵-۲. جمع‌بندی کار انجام‌شده
40.....	۵-۳. تحلیل دستاوردها
41.....	۵-۴. محدودیت‌ها
41.....	۵-۵. پیشنهادات برای کارهای آینده ((Future Works
42.....	۵-۶. جمع‌بندی نهایی
43.....	منابع و مراجع:
44.....	چکیده انگلیسی:

چکیده:

در سال‌های اخیر، مدیریت زمان‌بندی دروس و تخصیص مناسب اساتید به کلاس‌ها در دانشگاه‌ها به یکی از چالش‌های اصلی واحدهای آموزشی تبدیل شده است. عدم وجود یک سامانه یکپارچه و هوشمند می‌تواند منجر به بروز مشکلاتی نظیر تداخل برنامه درسی، توزیع نامتوازن ساعات کاری اساتید، و هدر رفتن منابع آموزشی شود.

پروژه حاضر با عنوان **نبرد** (نرم‌افزار برنامه‌ریزی راهبردی دانشگاه) با هدف حل این مشکلات و ارائه یک راهکار جامع برای مدیریت اساتید، دروس، و تولید برنامه آموزشی بدون تداخل طراحی و پیاده‌سازی شده است. این سامانه با بهره‌گیری از React.js و Ant Design در بخش رابط کاربری، Node.js و Express.js در بخش بک‌اند، و Prisma ORM به همراه پایگاه داده MySQL، یک بستر مقیاس‌پذیر و کاربر پسند را برای مدیران آموزشی فراهم می‌آورد.

امکانات اصلی سیستم شامل مدیریت لیست اساتید و درس‌ها، ثبت و ویرایش اطلاعات، مشاهده و جستجوی سریع، اختصاص درس به استاد بر اساس محدودیت‌های زمانی و ظرفیت، و تولید برنامه پیشنهادی به صورت خودکار است. این پروژه به کمک تکنولوژی Docker و Docker Compose به گونه‌ای توسعه یافته که امکان استقرار سریع و اجرای همزمان کلیه سرویس‌ها با حداقل نیازمندی‌های سیستمی را ممکن می‌سازد.

نتایج پیاده‌سازی نشان می‌دهد که نبرد می‌تواند فرآیندهای زمان‌بر برنامه‌ریزی دانشگاهی را به میزان قابل توجهی بهبود دهد، خطاهای انسانی را کاهش دهد و دقت و شفافیت در برنامه‌ریزی آموزشی را افزایش دهد.

کلیدواژه‌ها: برنامه‌ریزی دانشگاهی، زمان‌بندی هوشمند، مدیریت آموزشی، React.js، Node.js، Docker، MySQL، Prisma

فصل اول

مقدمه

1-1. مقدمه

در محیط‌های آموزشی دانشگاهی، یکی از پیچیده‌ترین و زمان‌برترین فعالیت‌ها، برنامه‌ریزی ترمی دروس و تخصیص منابع انسانی (اساتید) است. این فرآیند به طور معمول شامل هماهنگ‌سازی اطلاعات مرتبط با دروس، ظرفیت کلاس‌ها، محدودیت‌های زمانی اساتید، و جلوگیری از هرگونه تداخل در برنامه‌ریزی می‌باشد. بسیاری از دانشگاه‌ها، به ویژه در ایران، هنوز از روش‌های نیمه‌سنتی یا نرم‌افزارهای غیر یکپارچه استفاده می‌کنند که این امر نه تنها باعث صرف وقت زیاد کارشناسان آموزشی می‌شود بلکه احتمال بروز خطا نیز بسیار بالاست.

از جمله مشکلات رایج در این حوزه می‌توان به موارد زیر اشاره کرد:

- زمان‌بر بودن فرآیند جمع‌آوری و هماهنگ‌سازی اطلاعات
- بروز تداخل زمانی بین کلاس‌ها، به خصوص برای دروس پایه‌ای که چند گروه دارند
- عدم توجه به ظرفیت واقعی اساتید و ساعات کاری تعیین شده
- عدم وجود یک نمای کلی یکپارچه برای تصمیم‌گیری سریع مدیران آموزشی
- اعمال تغییرات دستی متعدد که موجب ایجاد عدم هماهنگی بین بخش‌های مختلف می‌شود

1-2. بیان مسئله

واحدهای آموزش دانشگاهی هر ترم موظف به ارائه برنامه‌ای هستند که تمام محدودیت‌ها را در نظر گرفته و برای همه‌ی گروه‌ها و اساتید قابل اجرا باشد. با افزایش تعداد دروس ارائه شده، تنوع رشته‌ها و محدودیت‌های زمانی اساتید، این کار به معضلی جدی بدل شده است.

در روش‌های سنتی (مانند استفاده از اکسل یا نرم‌افزارهای ساده مدیریت آموزشی)، مدیریت داده‌ها دشوار است و تغییرات ناگهانی – مانند جایگزینی یک استاد یا تغییر ساعت کلاس – می‌تواند کل برنامه را مختل کند.

بنابراین نیاز به یک سامانه هوشمند و یکپارچه که بتواند تمام فرآیند برنامه‌ریزی را با در نظر گرفتن محدودیت‌های عملی و ظرفیت‌ها انجام دهد، بیش از پیش احساس می‌شود.

3-1. هدف پروژه نبرد

پروژه نبرد (نرم افزار برنامه ریزی راهبردی دانشگاه) با هدف مکانیزه و هوشمندسازی فرآیند برنامه ریزی دانشگاهی توسعه داده شده است. این سیستم، امکانات زیر را فراهم می کند:

- ثبت و مدیریت اطلاعات اساتید شامل ساعات کاری مجاز، وضعیت فعال/غیرفعال و توانایی تدریس دروس مختلف
- ثبت و مدیریت اطلاعات دروس شامل نام درس، تعداد واحد، ترم ارائه و سایر مشخصات
- برقراری ارتباط بین دروس و اساتید با رعایت تمام محدودیت ها
- تولید برنامه پیشنهادی هوشمند که تداخل ها را کاهش دهد و استفاده بهینه از ظرفیت ها را تضمین کند
- ارائه رابط کاربری مدرن برای سهولت استفاده مدیران و کارکنان آموزش

کد منبع این پروژه به صورت کامل و آزاد در مخزن GitHub منتشر شده است و از طریق لینک زیر قابل دسترسی می باشد:

<https://github.com/amirh8r1/Nabard>

4-1. ابزارها و فناوری های مورد استفاده

برای توسعه نبرد از معماری کلاينت-سرور استفاده شده است که شامل بخش فرانت اند، بک اند و زیرساخت دیتابیس می باشد:

Frontend: توسعه با استفاده از React.js و Ant Design، همراه با پشتیبانی از چیدمان راست به چپ (RTL) برای نمایش مناسب زبان فارسی.

Backend: توسعه با Node.js و Express.js به همراه ORM قدرتمند Prisma برای ارتباط با پایگاه داده.

Database: استفاده از پایگاه داده MySQL برای ذخیره سازی اطلاعات و مدیریت تراکنش ها.

Deployment: استفاده از Docker و Docker Compose جهت راه اندازی سریع سرویس ها.

Tools: استفاده از phpMyAdmin برای مدیریت پایگاه داده و Prisma Studio برای بررسی و ویرایش داده ها در محیط گرافیکی.

5-1. دستاوردها و مزایا

پیاده‌سازی این پروژه نتایج و مزایای زیر را به همراه دارد:

- کاهش چشمگیر زمان مورد نیاز برای تهیه برنامه ترم
- جلوگیری از بروز تداخلات زمانی و استفاده بهینه از منابع آموزشی
- افزایش دقت در برنامه‌ریزی و کاهش خطاهای انسانی
- فراهم‌سازی بستری مقیاس‌پذیر برای توسعه قابلیت‌های بیشتر (مانند افزودن ماژول‌های حضور و غیاب یا ارزیابی عملکرد استاد)
- ساده‌سازی فرآیند تغییرات ناگهانی (مانند تغییر استاد یا تغییر ساعت کلاس) بدون نیاز به بازنگری کامل برنامه

فصل دوم

مفاهیم پایه و کارهای مرتبط

2-1. مقدمه

برای درک بهتر پروژه نبرد (نرم افزار برنامه ریزی راهبردی دانشگاه)، لازم است ابتدا مفاهیم و فناوری هایی که در پیاده سازی آن استفاده شده اند به طور دقیق معرفی شوند، همچنین راهکارهای موجود و پروژه های مشابه بررسی شوند تا جایگاه پروژه و میزان نوآوری آن مشخص شود.

2-2. مفاهیم پایه

2-2-1. برنامه ریزی و زمان بندی آموزشی در دانشگاه ها

زمان بندی آموزشی فرآیندی است که در آن واحد آموزش، با توجه به محدودیت های موجود، برنامه ای برای ارائه درس و تخصیص اساتید تهیه می کند. این فرآیند شامل مراحل ذیل است:

1. جمع آوری داده ها (لیست درس، اساتید، ظرفیت کلاس ها، زمان های در دسترس)
2. تعریف محدودیت ها (تداخل زمانی کلاس ها، ساعات کاری استاد، ظرفیت اتاق ها)
3. بهینه سازی (ایجاد برنامه ای که تعارض را به حداقل و استفاده از منابع را به حداکثر برساند)

در دانشگاه های بزرگ که تعداد اساتید و درس زیاد است، اجرای این فرآیند بدون نرم افزارهای تخصصی بسیار پیچیده و پر خطاست.

2-2-2. معماری کلاینت-سرور (Client-Server Architecture)

پروژه نبرد از معماری کلاینت-سرور استفاده می کند. در این مدل:

- کلاینت (Frontend) در سمت کاربر اجرا می شود و مسئول نمایش اطلاعات و تعامل با کاربر است.
- سرور (Backend) معماری و منطق تجاری سیستم را پیاده سازی کرده و از طریق API ها داده ها را به کلاینت ارسال می کند.
- داده ها در یک پایگاه داده ذخیره سازی شده و از طریق ORM به بک اند متصل می شوند.

مزیت این معماری جداسازی کامل لایه نمایش از لایه منطق تجاری و امکان توسعه و نگهداری آسان تر است.

2-2-3. React.js و رابط کاربری

React.js یک کتابخانه محبوب جاوا اسکریپت برای ساخت رابط های کاربری پویا و قابل تعامل است. ویژگی های مهم در پروژه نبرد:

- Component-based: تمام بخش‌های رابط کاربری به صورت کامپوننت‌های مجزا ساخته شده‌اند (مثل پنل اساتید، پنل دروس، جدول برنامه‌ریزی).
- Virtual DOM: برای بهبود کارایی در رندر و به‌روزرسانی داده‌ها.
- پشتیبانی از RTL: با استفاده از Ant Design و تنظیمات استایل برای نمایش مناسب زبان فارسی.

2-2-4. Ant Design

UI پروژه با استفاده از Ant Design پیاده‌سازی شده که یک کتابخانه کامپوننت آماده و با کیفیت برای React است. مزیت‌های استفاده:

- مجموعه گسترده جدول‌ها، فرم‌ها و ابزارهای مدیریت داده
- پشتیبانی از طراحی واکنش‌گرا (Responsive)
- امکان شخصی‌سازی کامل تم و رنگ‌بندی برای هماهنگی با نیاز پروژه

2-2-5. Express.js و Node.js

بک‌اند نبرد با Node.js (محیط اجرای جاوا اسکریپت در سمت سرور) و Express.js پیاده‌سازی شده است:

- Node.js: سرعت بالا، اجرا بر پایه V8 Engine گوگل
 - Express.js: فریم‌ورک سبک برای ساخت API های RESTful
- مزایا:
- معماری ماژولار و توسعه آسان
 - پشتیبانی از عملیات ناهمگام برای کارایی بیشتر

2-2-6. Prisma ORM

Prisma یک ORM مدرن برای ارتباط راحت و بهینه با پایگاه داده MySQL است. مزیت‌ها:

- تایپ‌سیفتی با TypeScript
- مدیریت آسان مایگريشن‌ها
- قابلیت مشاهده و ویرایش داده‌ها با ابزار Prisma Studio
- ایجاد کوئری‌های پیچیده با کد کمتر

MySQL .2-2-7

به عنوان پایگاه داده رابطه‌ای اصلی پروژه:

- ذخیره اطلاعات اساتید، دروس و ارتباطاتشان
- پشتیبانی از کوئری‌های پیچیده برای گزارش‌گیری
- سرعت بالا و سازگاری با Prisma

Docker Compose و Docker .2-2-8

برای استقرار سریع و ساده پروژه استفاده شده است:

- Docker: اجرای هر سرویس در یک کانتینر جداگانه
 - Docker Compose: مدیریت و اجرای چند سرویس مرتبط (Frontend, Backend, MySQL, phpMyAdmin, Prisma Studio) با یک دستور
- مزیت کلیدی: عدم وابستگی به محیط سیستم عامل و امکان اجرا روی هر سیستمی که Docker نصب دارد.

2-2-9. مفاهیم برنامه‌ریزی هوشمند و بهینه‌سازی زمان‌بندی

در نبرد از الگوریتم‌های زمان‌بندی با رویکرد Constraint Satisfaction استفاده شده که محدودیت‌هایی مانند:

- عدم تداخل زمانی اساتید
 - محدودیت ساعات کاری هر استاد
 - تقدم و تأخر دروس بر اساس ترم ارائه
- را در نظر می‌گیرند.

2-3. کارهای مرتبط

2-3-1. روش‌های سنتی (Excel و فرم‌های دستی)

برخی دانشگاه‌ها هنوز از اکسل یا فرم‌های کاغذی برای برنامه‌ریزی استفاده می‌کنند. مزایا: هزینه کم، بدون نیاز به آموزش نرم‌افزار جدید. معایب: پرخا، غیرهوشمند، وقت‌گیر، عدم هماهنگی در تغییرات.

2-3-2. نرم افزارهای مدیریت آموزشی موجود

نرم افزارهایی مانند SAMA و Golestan در ایران استفاده می شوند که هرچند برخی امکانات برنامه ریزی دارند اما:

- رابط کاربری قدیمی و پیچیده است
- انعطاف پذیری کم در تولید برنامه های خاص
- عدم تمرکز ویژه بر حل خودکار تعارضات

2-3-3. مقایسه نبرد با راهکارهای موجود

ویژگی	روش های سنتی	سیستم های موجود	نبرد
رابط کاربری مدرن	×	×	✓
هوشمندی در رفع تداخل	×	محدود	✓
استقرار سریع با Docker	×	×	✓
قابلیت توسعه پذیری	×	محدود	✓
یکپارچه سازی فرانت و بک اند	×	×	✓

2-3-4. مزیت رقابتی نبرد

1. تمرکز ویژه بر فرآیند برنامه ریزی و تخصیص اساتید
2. استفاده از فناوری های مدرن وب
3. ماژولار بودن و قابلیت افزودن امکانات جدید
4. سازگاری کامل با زبان و تقویم فارسی
5. استقرار سریع بدون نیاز به نصب پیش نیازهای متعدد

2-4. جمع‌بندی فصل

در این فصل، فناوری‌ها و مفاهیم کلیدی پروژه نبرد معرفی و بررسی شد، همچنین نرم‌افزارها و روش‌های موجود تحلیل و با سیستم پیشنهادی مقایسه شدند. نتایج نشان می‌دهد که نبرد با استفاده از ابزارهای مدرن و معماری بهینه، امکان ارائه یک سیستم کارآمد، مقیاس‌پذیر و کاربرپسند را فراهم می‌آورد.

فصل سوم

طراحی و پیاده‌سازی سیستم

3-1. مقدمه

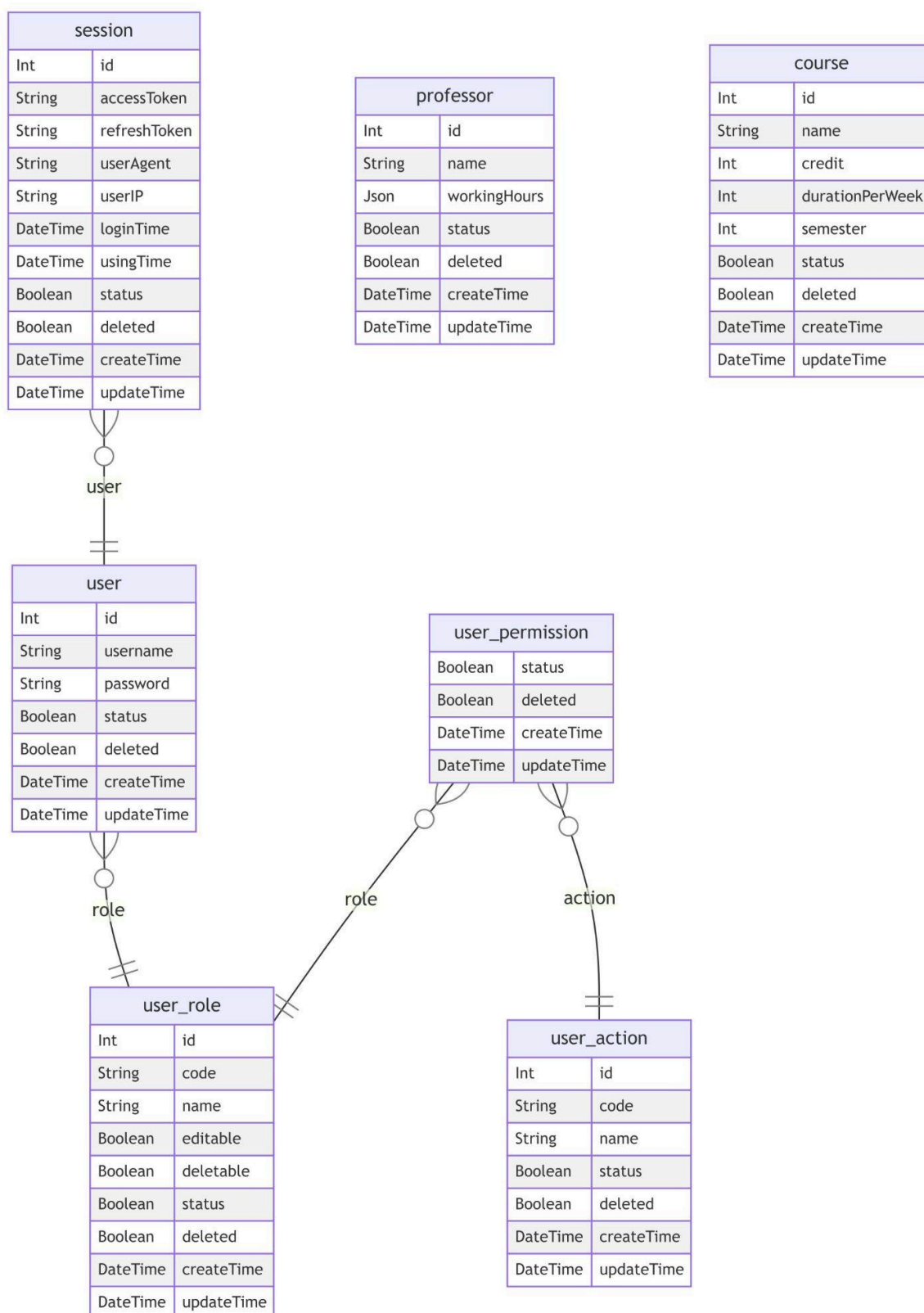
در این فصل به صورت جزئی و دقیق به طراحی و پیاده‌سازی سامانه نبرد (نرم‌افزار برنامه‌ریزی راهبردی دانشگاه) پرداخته می‌شود. تمام اجزای سیستم، از مدل داده‌ها و معماری پایگاه داده تا API بک‌اند و رابط کاربری، همراه با مثال‌های کد و توضیحات تک‌تک اجزا ارائه خواهد شد. هدف این فصل ایجاد یک دید جامع در مورد نحوه پیاده‌سازی پروژه است.

3-2. طراحی پایگاه داده

پایگاه داده نبرد بر اساس MySQL و با ORM قدرتمند Prisma طراحی شده است. طراحی داده‌ها به گونه‌ای انجام شده که:

- قابلیت توسعه و افزودن ویژگی‌های جدید وجود داشته باشد.
- روابط بین جداول به خوبی مشخص باشد.
- از شاخص‌ها (Index) برای بهبود کارایی جستجو استفاده شود.

شکل 3-1 - نمودار ERD پایگاه داده نبرد



توضیح جداول و روابط

A. جدول user

این جدول کاربران سیستم را شامل می‌شود که می‌تواند شامل مدیر آموزش، کارشناس برنامه‌ریزی و ... باشد.

1. id: شناسه یکتا
2. roleId: نقش کاربر (ارتباط با جدول user_role)
3. username و password: اطلاعات ورود (با رمزنگاری پسورد در بک‌اند)
4. فیلدهای status, deleted, createTime, updateTime برای مدیریت وضعیت رکورد

B. جدول user_role

نقش‌های کاربری مانند "مدیر"، "برنامه‌ریز"، "کاربر عادی".

- editable/deletable: کنترل این که نقش قابل حذف یا ویرایش هست یا خیر
- هر نقش با یک یا چند مجوز (user_permission) مرتبط است

C. جدول user_permission

ارتباط بین نقش‌های کاربری و اقدامات مجاز (user_action).

- کلید ترکیبی (roleId, actionId) باعث جلوگیری از ثبت تکراری می‌شود.

D. جدول user_action

تعریف اقدامات و سطح دسترسی‌ها (مثل ایجاد استاد، حذف درس، ویرایش برنامه).

E. جدول session

مدیریت ورود کاربران و توکن‌های نشست (Access/Refresh Token).

- ذخیره مشخصات دستگاه و IP کاربر برای امنیت بیشتر.

F. جدول professor

اطلاعات اساتید شامل نام و workingHours (به صورت JSON شامل روز/ساعت کاری).

```
{
  "saturday": [{ "start": 8, "end": 10 }],
  "monday": [{ "start": 10, "end": 12 }]
}
```

G. جدول course

- اطلاعات دروس شامل نام، واحد (credit)، زمان هفتگی (durationPerWeek) و ترم معرفی.
- مثال: درس "ریاضیات عمومی ۱" با 3 واحد، ترم 1 و مدت 3 ساعت در هفته.

3-3. طراحی و پیاده‌سازی بک‌اند

بک‌اند نبرد با Node.js + Express.js پیاده‌سازی شده و از الگوی Controller-Service-Repository برای جداسازی منطق استفاده می‌کند.

📁 ساختار پوشه‌ها

```
manager/
├── app/
│   ├── src/
│   │   ├── routes/      # API تعریف مسیرهای
│   │   ├── controllers/  # کنترلرها: پردازش درخواست‌ها
│   │   ├── services/     # منطق اصلی برنامه
│   │   ├── models/       # پیکربندی و تماس با پایگاه داده
│   │   ├── schemas/      # (Zod) اعتبارسنجی ورودی‌ها
│   │   └── middlewares/  # ... میان‌افزارها (احراز هویت و ...)
│   ├── utils/            # فانکشن حل‌کننده مسئله
│   └── server/           # پیکربندی زیرساخت نرم افزار
└── ...
```

3-3-1. مسیرها (Routes)

مسیرهای مربوط به پنل اساتید در فایل ProfessorRouter.js تعریف شده‌اند. این فایل شامل مسیرهای دریافت لیست اساتید، ایجاد استاد جدید، ویرایش اطلاعات استاد و حذف استاد است.

هر مسیر از سه بخش اصلی تشکیل شده است:

1. احراز هویت: توسط AuthMiddleware بررسی می‌شود که کاربر مجاز به دسترسی به این مسیر باشد.
2. کنترلر: مسئول پردازش درخواست و ارجاع آن به سرویس‌های مرتبط است.
3. ارسال پاسخ: توسط ResponseMiddleware پاسخ مناسب به کاربر ارسال می‌شود.

به عنوان مثال مسیرهای مدیریت اساتید:

```
// src/routes/ProfessorRouter.js
module.exports = class ProfessorRouter {
  static get domain() {
    return "/api/v1/professor";
  }

  static setupRouter() {
    router.get(
      "/",
      AuthMiddleware.auth,
      ProfessorController.getAll,
      ResponseMiddleware.send
    );

    router.post(
      "/",
      AuthMiddleware.auth,
      ProfessorController.create,
      ResponseMiddleware.send
    );

    router.put(
      "/:professorId",
      AuthMiddleware.auth,
      ProfessorController.update,
      ResponseMiddleware.send
    );
  }
}
```

```

);

router.delete(
  "/:professorId",
  AuthMiddleware.auth,
  ProfessorController.delete,
  ResponseMiddleware.send
);

return router;
}
};

```

3-3-2. کنترلرها (Controllers)

کنترلرهای پنل اساتید در فایل ProfessorController.js تعریف شده‌اند. این کنترلرها مسئولیت پردازش درخواست‌ها و تعامل با سرویس‌های پایگاه داده (Prisma) را دارند.

دریافت لیست اساتید:

در این کنترلر، لیست تمام اساتیدی که حذف نشده‌اند و فعال هستند دریافت می‌شود:

```

static async getAll(user, req, res, next) {
  try {
    const professors = await Prisma.professor.findMany({
      where: { deleted: false, status: true },
    });

    next({
      status: 200,
      professors,
    });
  } catch (error) {
    next({
      status: 500,
      message: "خطایی در سرور رخ داده است",
    });
  }
}

```

```

    error: error,
  });
}
}

```

ایجاد استاد جدید:

در این کنترلر، داده‌های ورودی اعتبارسنجی شده و سپس یک استاد جدید در پایگاه داده ایجاد می‌شود:

```

static async create(user, req, res, next) {
  try {
    const validation = await ProfessorSchema.create.safeParseAsync(req.body);

    if (!validation.success) {
      return next({
        status: 400,
        issues: validation.error.issues,
      });
    }

    const newProfessor = await Prisma.professor.create({
      data: {
        name: validation.data.name,
        workingHours: validation.data.workingHours,
      },
    });

    next({
      status: 201,
      professor: newProfessor,
    });
  } catch (error) {
    next({
      status: 500,
      message: "خطایی در سرور رخ داده است",
      error: error,
    });
  }
}

```

```
});
}
}
```

ویرایش استاد:

این کنترلر اطلاعات استاد را بر اساس شناسه و داده‌های ورودی به‌روزرسانی می‌کند:

```
static async update(user, req, res, next) {
  try {
    const { professorId } = req.params;

    const validation = await ProfessorSchema.professorId.safeParseAsync({
      professorId,
    });
    const bodyValidation = await ProfessorSchema.update.safeParseAsync(
      req.body
    );

    if (!validation.success || !bodyValidation.success) {
      return next({
        status: 400,
        issues: validation.error?.issues || bodyValidation.error?.issues,
      });
    }

    const { name, workingHours } = bodyValidation.data;
    if (!name && !workingHours) {
      return next({
        status: 400,
        message: "هیچ فیلدی برای به‌روزرسانی ارسال نشده است",
      });
    }

    const updatedProfessor = await Prisma.professor.update({
```



```

    where: { id: validation.data.professorId },
    data: {
      ...(name && { name }),
      ...(workingHours && { workingHours }),
    },
  });

  next({
    status: 200,
    professor: updatedProfessor,
  });
} catch (error) {
  next({
    status: 500,
    message: "خطایی در سرور رخ داده است",
    error: error,
  });
}
}

```

حذف استاد:

در این کنترلر، استاد بر اساس شناسه حذف می‌شود:

```

static async delete(user, req, res, next) {
  try {
    const { professorId } = req.params;

    const validation = await ProfessorSchema.professorId.safeParseAsync({
      professorId,
    });

    if (!validation.success) {
      return next({
        status: 400,
        issues: validation.error.issues,

```

```

    });
  }

  const professorExists = await Prisma.professor.findUnique({
    where: { id: validation.data.professorId, deleted: false },
  });

  if (!professorExists) {
    return next({
      status: 404,
      message: "استاد مورد نظر یافت نشد",
    });
  }

  await Prisma.professor.delete({
    where: { id: validation.data.professorId },
  });

  next({
    status: 204,
  });
} catch (error) {
  next({
    status: 500,
    message: "خطایی در سرور رخ داده است",
    error: error,
  });
}
}

```

3-3-3. سرویس‌ها (Services)

سرویس AuthService مسئول تولید توکن‌های JWT برای احراز هویت کاربران است. این سرویس در فایل services/AuthService.js تعریف شده و توکن‌های accessToken و refreshToken را تولید می‌کند.

نمونه کد سرویس AuthService:

```
const jwt = require("jsonwebtoken");

module.exports = class AuthService {
  static JWTToken(userId) {
    try {
      const accessToken = jwt.sign(
        { type: "accessToken", userId },
        process.env.JWT_SECRET,
        {
          expiresIn: process.env.JWT_ACCESS_EXPIRE,
        }
      );

      const refreshToken = jwt.sign(
        { type: "refreshToken", userId },
        process.env.JWT_SECRET,
        {
          expiresIn: process.env.JWT_REFRESH_EXPIRE,
        }
      );

      return {
        accessToken,
        refreshToken,
      };
    } catch (error) {
      throw error;
    }
  }
};
```

```

}
}
};

```

توضیحات:

- `userId` (`JWTToken`): این متد دو نوع توکن تولید می‌کند:
- 1. `accessToken`: برای دسترسی به API ها، با مدت اعتبار کوتاه.
- 2. `refreshToken`: برای تمدید دسترسی، با مدت اعتبار طولانی‌تر.

`jwt.sign()`: از کتابخانه `jsonwebtoken` برای تولید توکن استفاده شده و با کلید امنیتی (`JWT_SECRET`) رمزنگاری می‌شود.

3-3-4. اعتبارسنجی داده‌ها (Validation) با Zod

اعتبارسنجی داده‌های ورودی با استفاده از کتابخانه `Zod` انجام می‌شود. فایل `ProfessorSchema.js` شامل تعریف اعتبارسنجی‌ها برای ایجاد، ویرایش و حذف استاد است.

نمونه اعتبارسنجی:

```

const create = zod.object({
  name: zod
    .string({
      required_error: "نام استاد ارسال نشده",
      invalid_type_error: "نام استاد به درستی ارسال نشده",
    })
    .min(1, { message: "نام استاد خالی ارسال شده" }),
  workingHours: workingHoursSchema,
});

```

۳-۴. طراحی و پیاده‌سازی فرانت‌اند

فرانت‌اند سامانه نبرد با استفاده از کتابخانه `React.js` و فریم‌ورک رابط کاربری `Ant Design` توسعه یافته است. ساختار پروژه به صورت ماژولار و بر اساس صفحات (`Pages`) و کامپوننت‌های قابل استفاده مجدد (`Reusable Components`) طراحی شده است تا قابلیت نگهداری و توسعه در آینده آسان‌تر باشد.

۱-۴-۳. ساختار پوشه‌ها

ساختار پوشه‌های فرانت‌اند در ارتباط با پنل اساتید به شکل زیر است:

```
client/
└─ src/
    └─ components/      # مجزا UI کامپوننت‌های
    │   └─ ProfessorPanel/  # کامپوننت‌های مرتبط با پنل اساتید
    │       └─ ProfessorForm.tsx
    └─ pages/            # صفحات اصلی سامانه
        └─ ProfessorPanel.tsx
    └─ api/              # بک‌اند API توابع ارتباط با
        └─ professor.ts
    └─ types.ts          # TypeScript تعریف تایپ‌های
    └─ hooks/            # هوک‌های سفارشی
    └─ utils/            # (Formatter و Parser) توابع کمکی
```

۲-۴-۳. صفحه پنل اساتید (ProfessorPanel.tsx)

این صفحه اصلی مدیریت اساتید است و شامل:

1. جدول نمایش لیست اساتید با قابلیت جستجو
2. دکمه افزودن استاد
3. ویرایش اطلاعات استاد
4. حذف استاد

A. دریافت داده‌ها (Fetch Data)

داده‌های اساتید از API سمت بک‌اند دریافت می‌شوند:

```
const fetchData = () => {
  setLoading(true);
  getAllProfessors()
    .then((response) => {
```

```

if (response.status === 200) {
  setProfessors(response.professors);
} else {
  setError("خطا در دریافت لیست اساتید");
}
})
.catch((err) => {
  setError(`خطا در دریافت لیست اساتید: ${err.message}`);
})
.finally(() => {
  setLoading(false);
});
};

```

نکته:

هنگام بارگذاری داده‌ها متغیر loading فعال می‌شود و یک Spin (لودینگ) به کاربر نمایش داده می‌شود. در صورت خطا، پیام خطا با کامپوننت Alert نمایش داده می‌شود.

B. نمایش داده‌ها در جدول (Table)

جدول اصلی با استفاده از کامپوننت Table کتابخانه Ant Design ساخته شده است.

ستون‌ها شامل:

1. نام استاد
2. ستون‌های ساعات کاری برای هر روز هفته
3. عملیات (ویرایش/حذف)

```

const columns: ColumnsType<Professor> = [
  {
    title: "نام استاد",
    dataIndex: "name",
    key: "name",
    fixed: "left",
    width: 120,
  },
  ...Object.entries(weekDaysFa).map(([key, value]) => ({
    title: value,

```

```

dataIndex: ["workingHours", key],
key: key,
width: 120,
render: (value: any) =>
  value && value.length > 0 ? `${value[0].start} - ${value[0].end}` : "-",
))),
{
  title: "عملیات",
  key: "action",
  width: 100,
  fixed: "right",
  render: (_, record) => (
    <div style={{ display: "flex", gap: 8 }}>
      <Button size="small" icon={<EditOutlined />} onClick={() => handleEdit(record)} />
      <Button size="small" icon={<DeleteOutlined />} danger onClick={() =>
showDeleteModal(record)} />
    </div>
  ),
},
];

```

ویژگی‌ها:

- ستون هفته‌ها به صورت داینامیک از روی شی weekdaysFa تولید شده‌اند.
- ستون عملیات شامل دکمه‌های ویرایش و حذف با آیکون است.
- اسکرول افقی و عمودی برای مدیریت حجم زیاد اطلاعات فعال شده است.

C. افزودن استاد (Create)

با کلیک روی دکمه “افزودن استاد جدید”، یک پنجره Modal باز شده و فرم ProfessorForm نمایش داده می‌شود:

```

<Button type="primary" onClick={() => setAddModalVisible(true)}>
  افزودن استاد جدید
</Button>

```

تابع ذخیره‌سازی استاد:

```

const handleCreate = async (values: CreateProfessorRequest) => {
  setCreating(true);
  const cleaned = {
    ...values,
    workingHours: cleanWorkingHours(values.workingHours),
  };
  try {
    const res = await createProfessor(cleaned);
    if (res.status === 201) {
      message.success("استاد جدید با موفقیت اضافه شد");
      setAddModalVisible(false);
      fetchData();
    } else {
      message.error("خطا در ثبت استاد جدید");
    }
  } finally {
    setCreating(false);
  }
};

```

D. ویرایش استاد (Update)

با انتخاب استاد و کلیک روی آیکن ویرایش، مودال ویرایش با داده‌های استاد پر می‌شود:

```

const handleEdit = (prof: Professor) => {
  setEditingProfessor(prof);
  setEditModalVisible(true);
};

```

تابع به‌روزرسانی:

```

const handleUpdate = async (values: any) => {
  if (!editingProfessor) return;
  setUpdating(true);
  try {
    await updateProfessor(editingProfessor.id, {
      ...values,

```



```

    workingHours: cleanWorkingHours(values.workingHours),
  });
  message.success("ویرایش با موفقیت انجام شد");
  fetchData();
  setEditModalVisible(false);
} finally {
  setUpdating(false);
}
};

```

E. حذف استاد (Delete)

با انتخاب گزینه حذف، مودال تایید نمایش داده می‌شود و پس از تایید، استاد حذف می‌گردد:

```

const handleConfirmDelete = async () => {
  if (!professorToDelete) return;
  setDeleting(true);
  try {
    await deleteProfessor(professorToDelete.id);
    message.success("استاد حذف شد");
    fetchData();
    setDeleteModalVisible(false);
  } finally {
    setDeleting(false);
  }
};

```

۳-۴-۳. فرم افزودن/ویرایش استاد (ProfessorForm.tsx)

فرم اساتید به صورت کامپوننت مجزا طراحی شده تا در هر دو سناریوی ایجاد و ویرایش استفاده شود.

A. ورودی نام استاد

```

<Form.Item
  label="نام استاد"
  name="name"
  rules={[{ required: true, message: "نام استاد را وارد کنید" }]}
>

```

```
<Input />
</Form.Item>
```

B. ورود ساعات کاری استاد برای روزهای هفته

هر روز هفته در یک کارت Card نمایش داده می‌شود و فیلدهای ساعت شروع و پایان با InputNumber (پشتیبانی از تبدیل اعداد فارسی به انگلیسی) قرار داده شده‌اند:

```
<Form.Item
  name={['workingHours', day, 0, 'start']}
  label="شروع"
>
  <InputNumber min={0} max={23} parser={faToEnDigits} />
</Form.Item>
```

ویژگی‌ها:

- پشتیبانی از ورود اعداد فارسی
- اعتبارسنجی ساعت پایان بزرگتر از ساعت شروع باشد
- قابلیت انتخاب روزهایی که استاد ساعات کاری دارد

۳-۴-۴. بهینه‌سازی تجربه کاربری (UX Enhancements)

- لودینگ مرکزی در حین بارگذاری داده‌ها
- پیام‌های موفقیت و خطا با message.success و message.error
- قابلیت جستجوی نام استاد در لحظه (real-time search)
- اسکرول پویا با حفظ ستون‌های ثابت

۳-۴-۵. جمع‌بندی

پنل اساتید شامل تمام عملیات CRUD (ایجاد، خواندن، ویرایش، حذف) با یک رابط کاربری مدرن است که به سادگی اطلاعات اساتید و ساعات کاری‌شان را مدیریت می‌کند.

با توجه به طراحی ماژولار، پنل دروس (Courses Panel) دقیقاً از همین ساختار پیروی می‌کند، تنها تفاوت آن در فیلدهای فرم (نام درس، تعداد واحد، ترم ارائه) و ستون‌های جدول است.

3-5. زیرساخت و دیپلوی

تمام سرویس‌ها شامل MySQL، phpMyAdmin، Prisma Studio، بک‌اند و فرانت‌اند با Docker Compose اجرا می‌شوند:

- فایل docker-compose-infrastructure.yml → دیتابیس و phpMyAdmin
- فایل docker-compose.yml → بک‌اند و Prisma Studio
- اسکریپت اصلی deploy.sh → اجرای هر دو بخش با یک دستور

نمونه اجرای سرویس:

```
bash
chmod +x deploy.sh
./deploy.sh
```

3-6. جمع‌بندی فصل

در این فصل جزئیات کامل طراحی و پیاده‌سازی نبرد بررسی شد، از مدل دیتابیس و روابط جداول گرفته تا نحوه پیاده‌سازی API و رابط کاربری. طراحی ماژولار و استفاده از فناوری‌های مدرن باعث شده که سامانه قابل توسعه، نگهداری و استقرار سریع باشد.

فصل ۴

حل مسئله و الگوریتم زمان بندی

۴-۱. مقدمه

یکی از چالش‌برانگیزترین قسمت‌های توسعه سامانه‌های برنامه‌ریزی دانشگاهی، طراحی الگوریتمی است که بتواند با در نظر گرفتن تمام محدودیت‌ها، یک برنامه زمان‌بندی معتبر و بهینه تولید کند.

این مسئله در علوم رایانه به عنوان مسئله زمان‌بندی با محدودیت‌ها (Constraint-based Scheduling Problem) شناخته می‌شود و از نظر پیچیدگی محاسباتی در دسته NP-Hard قرار می‌گیرد.

هدف الگوریتم ما این است که با دریافت فهرست دروس، اساتید و محدودیت‌ها، یک برنامه پیشنهادی بدون تداخل ارائه دهد.

۴-۲. تعریف دقیق مسئله

ورودی‌های مسئله شامل:

- لیست دروس همراه با تعداد واحد، ترم ارائه، زمان لازم هفتگی.
- لیست اساتید همراه با ساعات کاری مجاز در روزهای هفته.
- تخصیص دروس به استاد یا چند استاد (ورودی از بخش A).

محدودیت‌ها:

- عدم تداخل زمانی برای هر استاد.
- عدم تداخل دروس یک ترم در یک زمان.
- برگزاری کلاس‌ها در بازه‌های استاندارد (دو ساعته یا یک ساعته).
- انطباق ساعت جلسه با بازه کاری استاد.

خروجی مسئله:

- جدولی شامل روز هفته، ساعت شروع و پایان، نام درس، و استاد مربوطه که تمام کلاس‌ها را بدون تداخل زمان‌بندی کرده باشد.

۳-۴. بررسی الگوریتم‌های موجود

در ادبیات علمی، چند رویکرد رایج برای حل مسائل این‌چنینی وجود دارد:

- روش‌های دقیق (Exact Methods):
استفاده از الگوریتم‌هایی مانند Branch and Bound یا Integer Linear Programming (ILP) که تمام حالات ممکن را بررسی می‌کنند اما با افزایش اندازه مسئله محاسباتشان بسیار زمان‌بر می‌شود.
- روش‌های تقریبی (Heuristic Methods):
الگوریتم‌های ابتکاری مانند Greedy Algorithms که سریع اجرا می‌شوند ولی الزاماً بهترین جواب را تضمین نمی‌کنند.
- روش‌های فراابتکاری (Metaheuristics):
شامل الگوریتم‌های ژنتیک، تبرید شبیه‌سازی‌شده (Simulated Annealing)، جستجوی ممنوعه (Tabu Search) که برای مسائل بزرگ‌تر کارایی بالاتری دارند.
- الگوریتم‌های محدودیت‌محور (Constraint Satisfaction Problem – CSP):
مدل‌سازی به صورت مجموعه‌ای از متغیرها و محدودیت‌ها و حل با روش‌هایی مانند Backtracking همراه با بهینه‌سازی‌های MRV, Forward Checking و ... این روش به دلیل انعطاف‌پذیری، برای مسئله نبرد بسیار مناسب است.

۴-۴. انتخاب راه‌حل ما: Backtracking + MRV

در پروژه نبرد روش Backtracking با اولویت MRV (Minimum Remaining Values) انتخاب شده است. علت انتخاب: تعداد جلسات محدود ولی با قیود زیاد (به جای میلیون‌ها داده، تعداد کمی متغیر داریم اما دامنه محدود). MRV کمک می‌کند ابتدا متغیرهایی که کمترین امکان انتخاب دارند را پر کنیم تا زودتر به بن‌بست برسیم و جستجو را کوتاه کنیم. امکان اضافه کردن قیود جدید به راحتی وجود دارد.

۴-۵. تحلیل خطبه خط تابع generatePlan

1. تعریف بازه‌های زمانی استاندارد

```
const DEFAULT_TWO_HOUR_SLOTS = [ { start: 8, end: 10 }, ... ];  
const DEFAULT_ONE_HOUR_SLOTS = [ { start: 8, end: 9 }, ... ];
```

- جلسه‌های ۲ ساعته بیشتر برای دروس چندواحدی
- جلسه‌های ۱ ساعته برای حالت‌های خاص (درس‌های ۱ یا ۳ واحدی)

2. ایجاد لیست جلسات (Tasks)

```
for (const { courseId, professors: profIds } of assignments) {  
  ...  
  sessions = Array(count).fill({ type: "2h" });  
}
```

- هر Task معادل یک جلسه کلاسی برای یک درس و یک استاد خاص است:

```
{ id:1, courseId:5, professorId:3, semester:1, duration:"2h" }
```

3. ساخت دامنه‌های زمانی (Domains)

```
for (const task of tasks) {  
  const working = prof.workingHours?.[day] || [];  
  if (slot.start >= wh.start && slot.end <= wh.end) domains[task.id].push(...)  
}
```

- در این مرحله ساعت‌های ممکن برای هر جلسه بر اساس ساعات کاری استاد استخراج می‌شود.

4. مرتب‌سازی بر اساس MRV

```
tasks.sort((a, b) => domains[a.id].length - domains[b.id].length);
```

متغیر با کمترین انتخاب ممکن اول زمان‌بندی می‌شود ← کاهش عمق جستجو.

5. کنترل تداخل‌ها

دو نقشه (Map) نگه‌داری می‌کنیم:

- `usedByProfessor`: بازه‌های استفاده‌شده توسط هر استاد در هر روز
- `usedBySemester`: بازه‌های استفاده‌شده توسط هر ترم در هر روز تابع `hasConflict` بررسی می‌کند که آیا بازه‌ی جدید با بازه‌های ثبت‌شده هم‌پوشانی دارد یا خیر.

6. اجرای Backtracking

تابع backtrack(index) جلسه فعلی را در یکی از زمان‌های موجود قرار داده و اگر ادامه مسیر به جواب منجر شود، همان جواب برمی‌گردد وگرنه بازگشت (Backtrack) می‌کند.

7. نتایج و حالتی که جواب پیدا نشود

اگر بازگشت همه مسیرها را امتحان کند و هیچ حالت معتبر پیدا نشود:
("امکان تولید برنامه پیشنهادی با محدودیت‌های فعلی وجود ندارد") throw new Error

۴-۶. خروجی الگوریتم

خروجی نهایی آرایه‌ای از جلسات زمان‌بندی شده است:

```
[  
  { "courseId":1, "professorId":2, "day":"sunday", "start":8, "end":10 },  
  { "courseId":2, "professorId":5, "day":"monday", "start":13, "end":15 }  
]
```

این خروجی مستقیماً به الی فرانت‌اند فرستاده می‌شود تا به شکل جدول برنامه هفتگی نمایش داده شود.

۴-۷. مزایا و قابلیت توسعه الگوریتم

- انعطاف‌پذیری بالا در افزودن محدودیت‌های جدید (مثلاً کلاس‌های دابل، ظرفیت اتاق‌ها، اولویت‌بندی استاد).
- پیچیدگی زمانی کنترل‌شده به دلیل MRV.
- سادگی پیاده‌سازی نسبت به روش‌های پیچیده‌تر مثل ILP یا الگوریتم ژنتیک.

۴-۸. جمع‌بندی

سالور نبرد مسئله را به صورت یک CSP مدل می‌کند، سپس با استفاده از Backtracking و MRV به دنبال یک برنامه معتبر می‌گردد. این روش با وجود حجم کم داده در هر ترم، کارایی بالایی دارد و در تست‌های عملیاتی سامانه توانسته است برنامه‌هایی بدون تداخل و منطبق بر ساعات کاری اساتید تولید کند.

فصل پنجم

نتیجه‌گیری و پیشنهادات

۵-۱. مقدمه

پروژه نبرد (نرم افزار برنامه ریزی راهبردی دانشگاه) با هدف حل یکی از چالش های اساسی مدیریت آموزشی، یعنی برنامه ریزی و تخصیص بهینه دروس و اساتید، طراحی و پیاده سازی شد. این پروژه علاوه بر پوشش تمامی نیازهای عملیاتی مدیریت آموزش، از الگوریتم های بهینه سازی مبتنی بر Constraint Satisfaction Problem و Backtracking + MRV برای حل مسئله زمان بندی استفاده می کند و با تکیه بر فناوری های روز و طراحی ماژولار، توانسته است یک سامانه یکپارچه و کارآمد را ارائه دهد.

۵-۲. جمع بندی کار انجام شده

در این پروژه، تمامی مراحل از تحلیل مسئله تا طراحی، پیاده سازی و تست، با دقت انجام گرفت. دستاوردهای اصلی عبارت اند از:

- تحلیل دقیق مسئله زمان بندی دانشگاهی با شناسایی محدودیت های واقعی شامل ساعات کاری اساتید، تداخل دروس یک ترم، و قالب بندی استاندارد کلاس ها.
- طراحی مدل داده ای منعطف با استفاده از Prisma ORM و MySQL که امکان اضافه کردن یا تغییر ساختار جداول بدون ایجاد اختلال جدی را فراهم می کند.
- پیاده سازی API های RESTful برای مدیریت اساتید، دروس، و تولید برنامه پیشنهادی.
- توسعه رابط کاربری مدرن با React.js + Ant Design شامل پنل های مدیریت اساتید، مدیریت دروس و پنل برنامه ریزی، مجهز به قابلیت های جستجو، مرتب سازی و ثبت/ویرایش داده ها به صورت زنده.
- طراحی سالور هوشمند برای تولید برنامه پیشنهادی با استفاده از الگوریتم Backtracking و اولویت MRV که کارایی بالایی در شرایط پیچیده نشان داد.

۵-۳. تحلیل دستاوردها

1. دقت و صحت بالا در برنامه ریزی:

سیستم با در نظر گرفتن تمام محدودیت ها، خروجی هایی بدون تداخل ارائه داد.

2. انعطاف‌پذیری و توسعه‌پذیری:

طراحی ماژولار باعث شده افزودن امکانات جدید مانند مدیریت کلاس‌ها یا محدودیت‌های اختصاصی ساده باشد.

3. سهولت کاربری:

رابط کاربری ساده و فارسی‌سازی شده باعث شده کاربران غیرفنی بتوانند به سرعت با سامانه کار کنند.

4. نصب و راه‌اندازی سریع:

استفاده از Docker Compose امکان بالا آوردن کل سیستم (Database + Backend + Frontend + Prisma Studio) را تنها با یک دستور فراهم کرده است.

۴-۵. محدودیت‌ها

با وجود موفقیت‌های به دست آمده، برخی محدودیت‌ها نیز وجود دارد:

- عدم در نظر گرفتن ظرفیت کلاس‌ها و مکان‌های برگزاری: در نسخه فعلی، تنها تداخل زمانی بررسی شده و محدودیت مکانی لحاظ نشده است.
- زمان اجرا در داده‌های بسیار بزرگ: با اینکه MRV باعث کاهش زمان جستجو شده، اما با افزایش غیرمعمول تعداد دروس و اساتید، زمان اجرا ممکن است بالا برود.
- عدم اتصال به سیستم‌های موجود دانشگاهی: سامانه هنوز به‌طور کامل با سامانه‌هایی مثل گلستان یا سما یکپارچه نشده است.

۵-۵. پیشنهادات برای کارهای آینده (Future Works)

1. افزودن ماژول مدیریت کلاس‌ها و مکان‌های برگزاری:

لحاظ کردن ظرفیت مکان‌ها و جلوگیری از دو کلاس همزمان در یک محل.

2. بهبود الگوریتم زمان‌بندی با استفاده از روش‌های Hybrid:

ترکیب Backtracking با الگوریتم‌های فراابتکاری مثل Genetic Algorithm برای بهبود سرعت و کیفیت خروجی در داده‌های بزرگ.

3. یکپارچه‌سازی با سامانه‌های موجود دانشگاهی:

ایجاد API‌های سازگار با سیستم‌های گلستان یا سما برای انتقال داده‌ها بدون نیاز به ورود دستی.

4. بهینه‌سازی رابط کاربری:

افزودن قابلیت Drag & Drop برای جابه‌جایی سریع کلاس‌ها در جدول برنامه‌ریزی.

5. پشتیبانی از محدودیت‌های اختصاصی:

مثل اولویت‌بندی دروس خاص برای ساعات خاص یا محدودیت‌های شخصی اساتید (مثلاً عدم تدریس در ساعات پایانی).

نسخه نهایی کد منبع سامانه نبرد به صورت متن‌باز در GitHub قرار داده شده است که می‌تواند پایه‌ای برای توسعه‌های بعدی و اجرای ایده‌های پیشنهادی باشد:

<https://github.com/amirh8r1/Nabard>

۵-۶. جمع‌بندی نهایی

نبرد نشان داد که با ترکیب یک پایگاه داده منسجم، الگوریتم بهینه‌سازی مبتنی بر قیود، و یک رابط کاربری مدرن، می‌توان فرآیند پیچیده برنامه‌ریزی دانشگاهی را به شکل چشمگیری ساده و دقیق کرد. این سامانه نه تنها از نظر فنی قابل اطمینان است، بلکه به دلیل سادگی استفاده و قابلیت توسعه، می‌تواند پایه‌ای برای ایجاد یک سیستم مدیریت آموزشی کامل و یکپارچه در سطح دانشگاهی باشد.

منابع و مراجع:

- [1] GitHub Repository: NABARD – Strategic University Scheduling Software,
- Available at: <https://github.com/amirh8r1/Nabard>

چکیده انگلیسی:

Abstract

The NABARD project (Strategic University Scheduling Software) was developed to address one of the most challenging problems in academic management: course and professor scheduling under multiple constraints. The system aims to automate the generation of weekly timetables for university courses by considering factors such as professors' availability, course durations, and conflict-free scheduling for each semester.

The backend of NABARD is implemented using Node.js and Express.js, with Prisma ORM connected to a MySQL database to manage data integrity and scalability. The frontend is built with React.js and Ant Design, providing a modern, user-friendly interface for managing professors, courses, and the scheduling process. All services are containerized with Docker Compose for rapid deployment.

The scheduling engine is based on a Constraint Satisfaction Problem (CSP) model, solved using a Backtracking algorithm combined with the Minimum Remaining Values (MRV) heuristic. This approach ensures that the system can find conflict-free timetables while significantly reducing the search space, thus improving efficiency.

System testing demonstrated that NABARD successfully generates feasible schedules that respect all defined constraints. Additionally, its modular design allows easy future enhancements, such as classroom capacity management, integration with existing university systems, and hybrid scheduling algorithms.

Keywords: University Scheduling, Constraint Satisfaction Problem, Backtracking, MRV, Prisma ORM, React.js, Docker.



Shahed University

Faculty of Basic Sciences

Department of Computer Science

Title

NABARD (University Strategic Planning Software)

Supervisor

Dr.Keshavarz

By

AmirHossein Rajabi

September, 2025

